

Masooma Batool School — Python Streamlit App

Source code for app.py. The application is written in Python and uses Streamlit.

```
import streamlit as st
import pandas as pd
from datetime import datetime, date
from io import BytesIO
import base64
import hashlib

# =====
# SCHOOL MANAGEMENT WEB APP
# Pure Python application: Streamlit
# Save this file as app.py
# Run: streamlit run app.py
#
# Note:
# - No HTML/C/C++/Java is written in this project.
# - Replace SAMPLE_SCHOOL_IMAGE with the school's own licensed
# photo when you have it.
# =====
st.set_page_config(
    page_title="Masooma Batool School Portal",
    page_icon="🎓",
    layout="wide",
    initial_sidebar_state="expanded"
)
SCHOOL_NAME = "Masooma Batool School"
CONTACT_PERSON = "Madam Kanam Masooma Batool"
PHONE = "03308995876"
# Public sample school image. Replace with your school's own photo/logo.
SAMPLE_SCHOOL_IMAGE = (
    "https://images.unsplash.com/photo-1562774053-701939374585"
    "?auto=format&fit=crop&w=1600&q=80"
)
# ----- SAMPLE DATA -----
if "students" not in st.session_state:
    st.session_state.students = pd.DataFrame([
        {"Roll No": "1001", "Name": "Ali Ahmed", "Class": "9", "Section": "A", "Parent": "Ahmed Khan", "Fee": "Paid"},
        {"Roll No": "1002", "Name": "Ayesha Noor", "Class": "9", "Section": "A", "Parent": "Noor Ahmad", "Fee": "Due"},
        {"Roll No": "1003", "Name": "Hassan Raza", "Class": "8", "Section": "B", "Parent": "Raza Ali", "Fee": "Paid"},
    ])
if "staff" not in st.session_state:
    st.session_state.staff = pd.DataFrame([
        {"Name": "Madam Kanam Masooma Batool", "Role": "Principal", "Subject": "Administration"},
        {"Name": "Muhammad Usman", "Role": "Teacher", "Subject": "Mathematics"},
        {"Name": "Sadia Bibi", "Role": "Teacher", "Subject": "English"},
        {"Name": "Imran Khan", "Role": "Librarian", "Subject": "Library"},
    ])
if "notices" not in st.session_state:
    st.session_state.notices = [
        "Welcome to the School Management Portal.",
        "Parent-teacher meeting will be announced through the notice board.",
        "Students should check the timetable and homework regularly."
    ]
if "attendance" not in st.session_state:
    st.session_state.attendance = pd.DataFrame([
        {"Date": str(date.today()), "Roll No": "1001", "Status": "Present"},
        {"Date": str(date.today()), "Roll No": "1002", "Status": "Absent"},
        {"Date": str(date.today()), "Roll No": "1003", "Status": "Present"},
    ])
if "fees" not in st.session_state:
    st.session_state.fees = pd.DataFrame([
        {"Roll No": "1001", "Student": "Ali Ahmed", "Month": "September", "Amount": 5000, "Status": "Paid"},
        {"Roll No": "1002", "Student": "Ayesha Noor", "Month": "September", "Amount": 5000, "Status": "Due"},
        {"Roll No": "1003", "Student": "Hassan Raza", "Month": "September", "Amount": 5000, "Status": "Paid"},
    ])
if "books" not in st.session_state:
    st.session_state.books = pd.DataFrame([
        {"Book ID": "B001", "Title": "Oxford English", "Category": "English", "Status": "Available"},
        {"Book ID": "B002", "Title": "Mathematics 9", "Category": "Mathematics", "Status": "Issued"},
        {"Book ID": "B003", "Title": "General Science", "Category": "Science", "Status": "Available"},
    ])
if "assignments" not in st.session_state:
    st.session_state.assignments = pd.DataFrame([
        {"Subject": "Mathematics", "Title": "Algebra Exercise", "Due Date": "2026-09-25"},
        {"Subject": "English", "Title": "Essay Writing", "Due Date": "2026-09-27"},
    ])
# ----- HELPERS -----
def simple_password_hash(password):
    return hashlib.sha256(password.encode()).hexdigest()
DEMO_USERS = {
    "admin": ("admin123", "Admin"),
    "principal": ("principal123", "Principal"),
    "teacher": ("teacher123", "Teacher"),
    "student": ("student123", "Student"),
    "parent": ("parent123", "Parent / Guardian"),
    "clerk": ("clerk123", "Clerk / Accounts"),
    "librarian": ("librarian123", "Librarian"),
}
def login_screen():
    st.title("🎓 School Portal Login")
    st.caption("Demo accounts are included for testing. Change passwords before real deployment.")
    username = st.text_input("Username")
    password = st.text_input("Password", type="password")
    if st.button("Login", type="primary"):
        if username in DEMO_USERS and DEMO_USERS[username][0] == password:
            st.session_state.logged_in = True
            st.session_state.username = username
            st.session_state.role = DEMO_USERS[username][1]
```

```

st.rerun()
else:
st.error("Invalid username or password.")
def metric_card(label, value):
st.metric(label, value)
def export_csv(df):
return df.to_csv(index=False).encode("utf-8")
def make_result_pdf(student_name, roll_no, marks):
# Generates a simple downloadable result PDF from Python.
from reportlab.lib.pagesizes import A4
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle
from reportlab.lib import colors
from reportlab.lib.styles import getSampleStyleSheet
buffer = BytesIO()
doc = SimpleDocTemplate(buffer, pagesize=A4)
styles = getSampleStyleSheet()
story = [
Paragraph(SCHOOL_NAME, styles["Title"]),
Paragraph("Student Progress Report", styles["Heading2"]),
Spacer(1, 12),
Paragraph(f"Student: {student_name}", styles["Normal"]),
Paragraph(f"Roll No: {roll_no}", styles["Normal"]),
Spacer(1, 12),
]
rows = [{"Subject", "Marks", "Total"}]
total = 0
for subject, mark in marks.items():
rows.append([subject, str(mark), "100"])
total += int(mark)
rows.append(["Total", str(total), str(len(marks) * 100)])
table = Table(rows, colWidths=[220, 100, 100])
table.setStyle(TableStyle([
("BACKGROUND", (0, 0), (-1, 0), colors.lightgrey),
("GRID", (0, 0), (-1, -1), 0.7, colors.black),
("ALIGN", (1, 1), (-1, -1), "CENTER"),
("FONTNAME", (0, 0), (-1, 0), "Helvetica-Bold"),
]))
story.append(table)
story.append(Spacer(1, 20))
story.append(Paragraph(f"Contact: {PHONE}", styles["Normal"]))
story.append(Paragraph(f"Contact Person: {CONTACT_PERSON}", styles["Normal"]))
doc.build(story)
buffer.seek(0)
return buffer
# ----- PUBLIC WEBSITE -----
def public_home():
st.image(SAMPLE_SCHOOL_IMAGE, use_container_width=True)
st.title("■ Masooma Batool School")
st.subheader("Learning • Character • Excellence")
st.write(
"Welcome to our school portal. This public website provides school "
"information, admissions, notices, events, facilities and contact details."
)
col1, col2, col3 = st.columns(3)
with col1:
st.info("■ Quality Education")
with col2:
st.info("■ Labs & Library")
with col3:
st.info("■ Sports & Activities")
st.subheader("Principal's Message")
st.write(
"Our aim is to provide a safe, respectful and productive learning "
"environment where students can develop academically and personally."
)
def public_pages(page):
if page == "Home":
public_home()
elif page == "About School":
st.header("About School")
st.write("School introduction, mission, vision and academic information.")
st.write("Facilities include classrooms, science/computer labs, library, playground and activity areas.")
elif page == "Faculty & Staff":
st.header("Faculty / Staff")
st.dataframe(st.session_state.staff, use_container_width=True, hide_index=True)
elif page == "Facilities":
st.header("School Facilities")
cols = st.columns(4)
for c, text in zip(cols, ["■ Classrooms", "■ Labs", "■ Library", "■ Playground"]):
c.success(text)
elif page == "News & Notices":
st.header("News, Notices & Events")
for notice in st.session_state.notices:
st.info(notice)
elif page == "Gallery":
st.header("Gallery")
st.image(SAMPLE_SCHOOL_IMAGE, caption="Sample school image")
st.caption("Use only images you own or have permission to publish.")
elif page == "Admissions":
st.header("Online Admission Form")
with st.form("admission_form"):
name = st.text_input("Student Name")
father = st.text_input("Father / Guardian Name")
dob = st.date_input("Date of Birth", value=date(2012, 1, 1))
grade = st.selectbox("Applying Class", ["Playgroup", "1", "2", "3", "4", "5", "6", "7", "8", "9", "10"])
phone = st.text_input("Parent Contact Number")
documents = st.file_uploader(
"Upload documents (B-Form, photo, certificates)",
accept_multiple_files=True
)
submitted = st.form_submit_button("Submit Admission Form")
if submitted:
st.success(f"Application received for {name}.")
st.write("Application status: Under Review")

```

```

if documents:
    st.write(f"{len(documents)} document(s) attached.")
elif page == "Downloads":
    st.header("Downloads")
    for filename, content in [
        ("Prospectus.txt", "School prospectus placeholder"),
        ("Syllabus.txt", "Syllabus placeholder"),
        ("Date-Sheet.txt", "Date sheet placeholder"),
        ("Admission-Form.txt", "Admission form placeholder"),
    ]:
        st.download_button(
            label=f"■ {filename}",
            data=content,
            file_name=filename,
            mime="text/plain"
        )
    elif page == "Contact":
        st.header("Contact Us")
        st.write(f"***Contact Person:** {CONTACT_PERSON}")
        st.write(f"***Phone:** {PHONE}")
        st.write(f"***Email:** school@example.com")
        st.write(f"***Location:** Pakistan")
        st.map(pd.DataFrame({"lat": [24.8607], "lon": [67.0011]}))
        with st.form("contact_form"):
            name = st.text_input("Your Name")
            email = st.text_input("Email")
            message = st.text_area("Message")
            if st.form_submit_button("Send Message"):
                st.success("Your message has been recorded.")
# ----- DASHBOARDS -----
def admin_dashboard():
    st.header("■ Admin Dashboard")
    a, b, c, d = st.columns(4)
    a.metric("Students", len(st.session_state.students))
    b.metric("Staff", len(st.session_state.staff))
    c.metric("Fee Records", len(st.session_state.fees))
    d.metric("Notices", len(st.session_state.notices))
    tabs = st.tabs([
        "Students", "Staff", "Classes & Subjects",
        "Attendance", "Exams & Results", "Fees",
        "Notices", "Reports", "Backup & Audit"
    ])
    with tabs[0]:
        st.subheader("Student Management")
        st.dataframe(st.session_state.students, use_container_width=True, hide_index=True)
        with st.form("add_student"):
            roll = st.text_input("Roll No")
            name = st.text_input("Student Name")
            cls = st.text_input("Class")
            sec = st.text_input("Section")
            parent = st.text_input("Parent / Guardian")
            if st.form_submit_button("Add Student"):
                if roll and name:
                    new = pd.DataFrame([
                        {"Roll No": roll, "Name": name, "Class": cls,
                         "Section": sec, "Parent": parent, "Fee": "Due"}
                    ])
                    st.session_state.students = pd.concat(
                        [st.session_state.students, new], ignore_index=True
                    )
                    st.success("Student added.")
                    st.rerun()
                st.download_button(
                    "Download Student CSV",
                    export_csv(st.session_state.students),
                    "students.csv",
                    "text/csv"
                )
    with tabs[1]:
        st.subheader("Staff Management")
        st.dataframe(st.session_state.staff, use_container_width=True, hide_index=True)
    with tabs[2]:
        st.subheader("Classes, Sections & Subjects")
        st.write("Classes: Playgroup to Grade 10")
        st.write("Sections: A, B, C")
        st.write("Subjects: English, Urdu, Mathematics, Science, Islamiat, Computer, Social Studies")
    with tabs[3]:
        st.subheader("Attendance")
        st.dataframe(st.session_state.attendance, use_container_width=True, hide_index=True)
        if st.button("Add Today's Sample Attendance"):
            st.session_state.attendance = pd.concat([
                st.session_state.attendance,
                pd.DataFrame([
                    {"Date": str(date.today()),
                     "Roll No": "1001",
                     "Status": "Present"}
                ])
            ], ignore_index=True)
            st.rerun()
    with tabs[4]:
        st.subheader("Exams & Results")
        st.write("Exam schedule, marks entry, grades, result cards and analytics.")
        marks = {"English": 78, "Mathematics": 84, "Science": 81, "Urdu": 76, "Computer": 90}
        pdf = make_result_pdf("Ali Ahmed", "1001", marks)
        st.download_button(
            "■ Generate Result Card PDF",
            data=pdf,
            file_name="result_card_1001.pdf",
            mime="application/pdf"
        )
    with tabs[5]:
        st.subheader("Fees Management")
        st.dataframe(st.session_state.fees, use_container_width=True, hide_index=True)
        paid = int((st.session_state.fees["Status"] == "Paid").sum())

```

```

due = int((st.session_state.fees["Status"] == "Due").sum())
x, y = st.columns(2)
x.metric("Paid Records", paid)
y.metric("Due Records", due)
with tabs[6]:
    st.subheader("Publish Notice")
    notice = st.text_area("Notice / Announcement")
    if st.button("Publish Notice"):
        if notice.strip():
            st.session_state.notices.insert(0, notice.strip())
            st.success("Notice published.")
            st.rerun()
with tabs[7]:
    st.subheader("Reports & Analytics")
    x, y = st.columns(2)
    x.bar_chart(st.session_state.students["Class"].value_counts())
    y.bar_chart(st.session_state.attendance["Status"].value_counts())
with tabs[8]:
    st.subheader("Backup & Audit Logs")
    st.info("Production version should use a real database, automated encrypted backups and server-side audit logging.")
    st.download_button(
        "Backup Students CSV",
        export_csv(st.session_state.students),
        "school_backup_students.csv",
        "text/csv"
    )
    st.write("Audit example: Admin viewed Student Management.")
def student_dashboard():
    st.header("■ Student Module")
    st.write("**Student:** Ali Ahmed | **Roll No:** 1001 | **Class:** 9-A")
    a, b, c = st.columns(3)
    a.metric("Attendance", "92%")
    b.metric("Average Result", "82%")
    c.metric("Fee", "Paid")
    st.subheader("Timetable")
    st.dataframe(pd.DataFrame([
        {"Period": 1, "Subject": "Mathematics"},
        {"Period": 2, "Subject": "English"},
        {"Period": 3, "Subject": "Science"},
        {"Period": 4, "Subject": "Computer"},
    ]), use_container_width=True, hide_index=True)
    st.subheader("Homework / Assignments")
    st.dataframe(st.session_state.assignments, use_container_width=True, hide_index=True)
def teacher_dashboard():
    st.header("■■■■ Teacher Module")
    st.write("Daily attendance, marks, assignments, lesson plans and parent meetings.")
    st.subheader("Class Attendance")
    edited = st.data_editor(st.session_state.attendance, use_container_width=True, hide_index=True)
    if st.button("Save Attendance"):
        st.session_state.attendance = edited
        st.success("Attendance saved.")
    st.subheader("Assignments")
    st.dataframe(st.session_state.assignments, use_container_width=True, hide_index=True)
def parent_dashboard():
    st.header("■■■■■ Parent / Guardian Module")
    st.write("Linked Student: Ali Ahmed | Class: 9-A")
    a, b, c = st.columns(3)
    a.metric("Attendance", "92%")
    b.metric("Result", "82%")
    c.metric("Fee Status", "Paid")
    st.subheader("Notices & Teacher Messages")
    for n in st.session_state.notices:
        st.info(n)
    st.subheader("Online Leave Application")
    reason = st.text_area("Reason for leave")
    if st.button("Submit Leave Application"):
        st.success("Leave application submitted for review.")
def clerk_dashboard():
    st.header("■ Clerk / Accounts")
    st.dataframe(st.session_state.fees, use_container_width=True, hide_index=True)
    st.metric("Total Fee Due", int(
        st.session_state.fees.loc[st.session_state.fees["Status"] == "Due", "Amount"].sum()
    ))
def librarian_dashboard():
    st.header("■ Librarian")
    st.dataframe(st.session_state.books, use_container_width=True, hide_index=True)
    st.subheader("Issue / Return")
    book_id = st.text_input("Book ID")
    action = st.selectbox("Action", ["Issue", "Return"])
    if st.button("Save Library Transaction"):
        st.success(f"{action} transaction saved for {book_id}.")
def principal_dashboard():
    st.header("■ Principal / Vice Principal Dashboard")
    a, b, c, d = st.columns(4)
    a.metric("Enrollment", len(st.session_state.students))
    b.metric("Staff", len(st.session_state.staff))
    c.metric("Attendance Records", len(st.session_state.attendance))
    d.metric("Fee Due", int(st.session_state.fees.loc[st.session_state.fees["Status"] == "Due", "Amount"].sum()))
    st.subheader("School Overview")
    st.dataframe(st.session_state.students, use_container_width=True, hide_index=True)
# ----- APP -----
if "logged_in" not in st.session_state:
    st.session_state.logged_in = False
st.sidebar.title("■ School Portal")
mode = st.sidebar.radio("Portal", ["Public Website", "Login / Dashboard"])
if mode == "Public Website":
    page = st.sidebar.selectbox(
        "Public Pages",
        ["Home", "About School", "Faculty & Staff", "Facilities",
        "News & Notices", "Gallery", "Admissions", "Downloads", "Contact"]
    )
    public_pages(page)
else:
    if not st.session_state.logged_in:

```

```
login_screen()
else:
    st.sidebar.success(f"Logged in: {st.session_state.role}")
    if st.sidebar.button("Logout"):
        st.session_state.logged_in = False
    st.rerun()
    role = st.session_state.role
    if role == "Admin":
        admin_dashboard()
    elif role == "Principal":
        principal_dashboard()
    elif role == "Teacher":
        teacher_dashboard()
    elif role == "Student":
        student_dashboard()
    elif role == "Parent / Guardian":
        parent_dashboard()
    elif role == "Clerk / Accounts":
        clerk_dashboard()
    elif role == "Librarian":
        librarian_dashboard()
    st.sidebar.divider()
    st.sidebar.caption(f"Contact: {CONTACT_PERSON}")
    st.sidebar.caption(f"Phone: {PHONE}")
```